

Convolutional Neural Networks

Inner Products

Vector Inner Product

- $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 5 \\ 6 \\ 7 \\ 8 \end{bmatrix}$.

- Inner product:

$$\langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a}^T \mathbf{b} = \sum_i a_i b_i = 70.$$

Matrix Inner Product

- $\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix}$.

- Inner product:

$$\langle \mathbf{A}, \mathbf{B} \rangle = \sum_i \sum_j a_{ij} b_{ij} = 70.$$

- Property: $\langle \mathbf{A}, \mathbf{B} \rangle = \langle \text{vec}(\mathbf{A}), \text{vec}(\mathbf{B}) \rangle$.

Tensor Inner Product

• $\mathbf{A} =$  and $\mathbf{B} =$  .

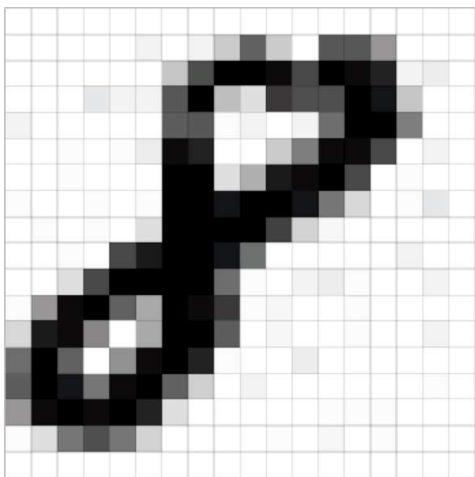
• Inner product:

$$\langle \mathbf{A}, \mathbf{B} \rangle = \sum_i \sum_j \sum_k a_{ijk} b_{ijk} .$$

• Property: $\langle \mathbf{A}, \mathbf{B} \rangle = \langle \text{vec}(\mathbf{A}), \text{vec}(\mathbf{B}) \rangle$.

Convolution for Images

An Image is A Matrix/Tensor of Pixel Values



The MNIST Dataset

- Each grayscale image, $\mathbf{x}_j$, is a 28×28 matrix.
- (Previously, we reshape it to a 784-dim vector.)

Convolution

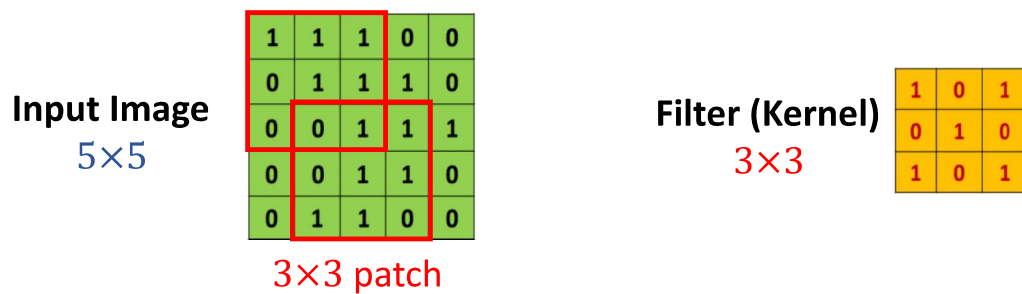
Input Image
5×5

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filter (Kernel)
3×3

1	0	1
0	1	0
1	0	1

Convolution



Question: how many 3×3 patches does a 5×5 image have?

Answer: $(5 - 3 + 1) \times (5 - 3 + 1) = 9$.

Convolution

Input Image
5×5

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filter (Kernel)
3×3

1	0	1
0	1	0
1	0	1

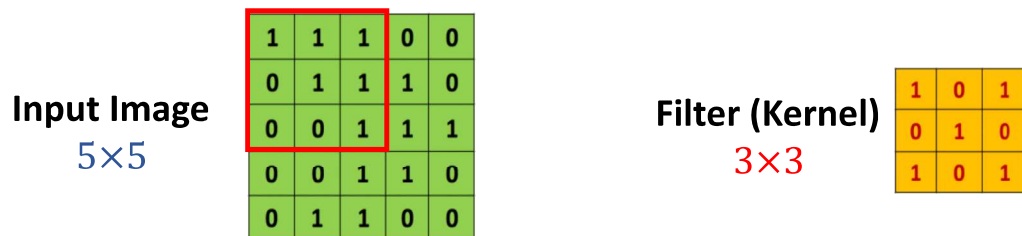


Convolution:

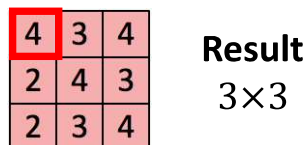
4	3	4
2	4	3
2	3	4

Result
3×3

Convolution



Convolution:



The value **4** is the inner product of the patch

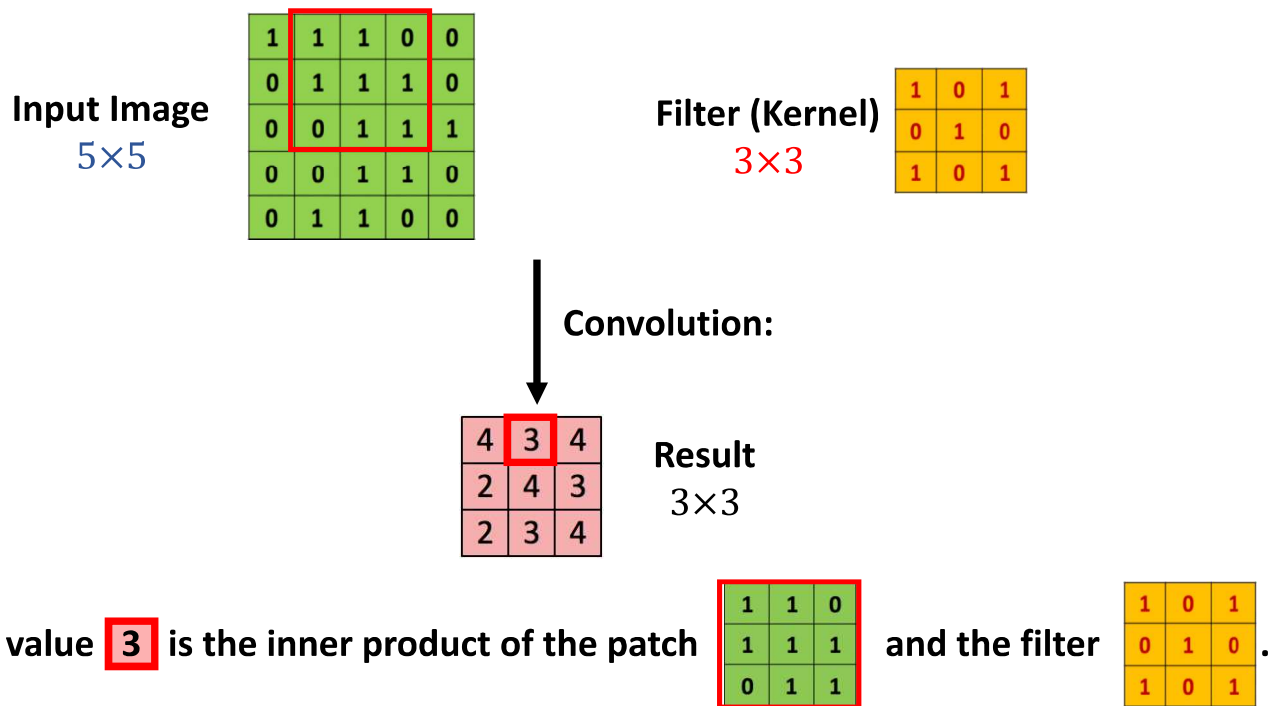
1	1	1
0	1	1
0	0	1

and the filter

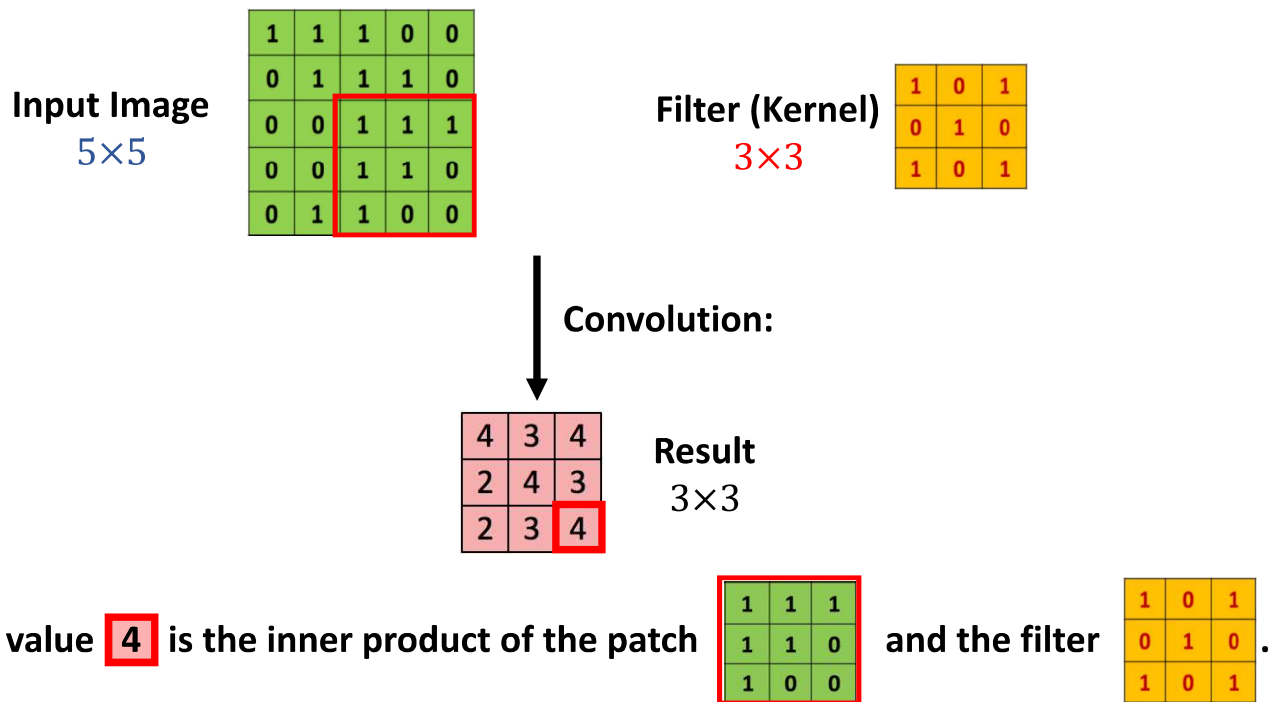
1	0	1
0	1	0
1	0	1

.

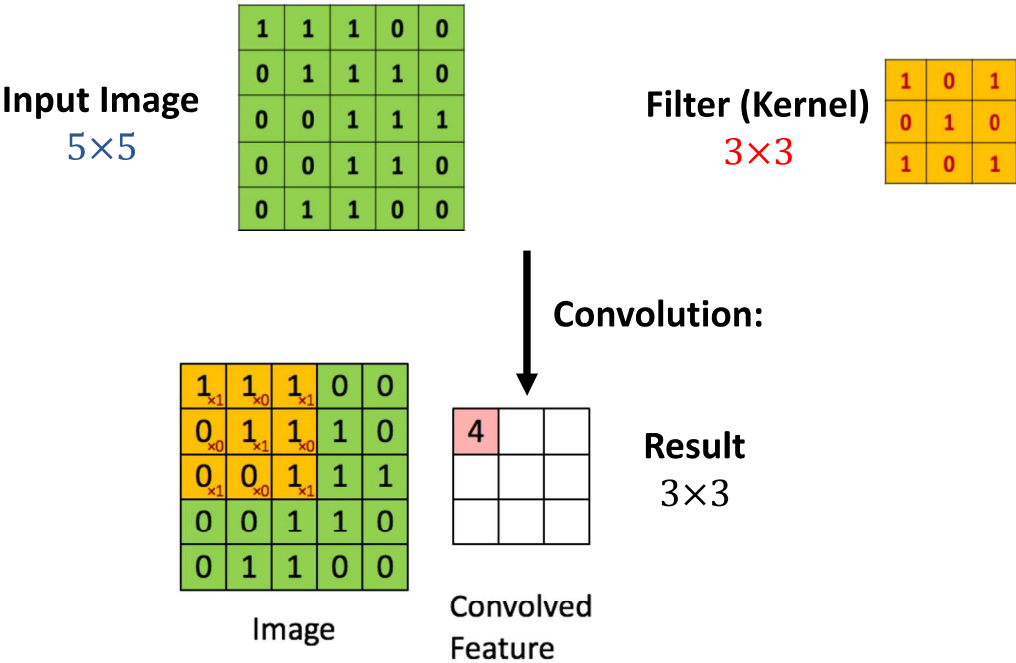
Convolution



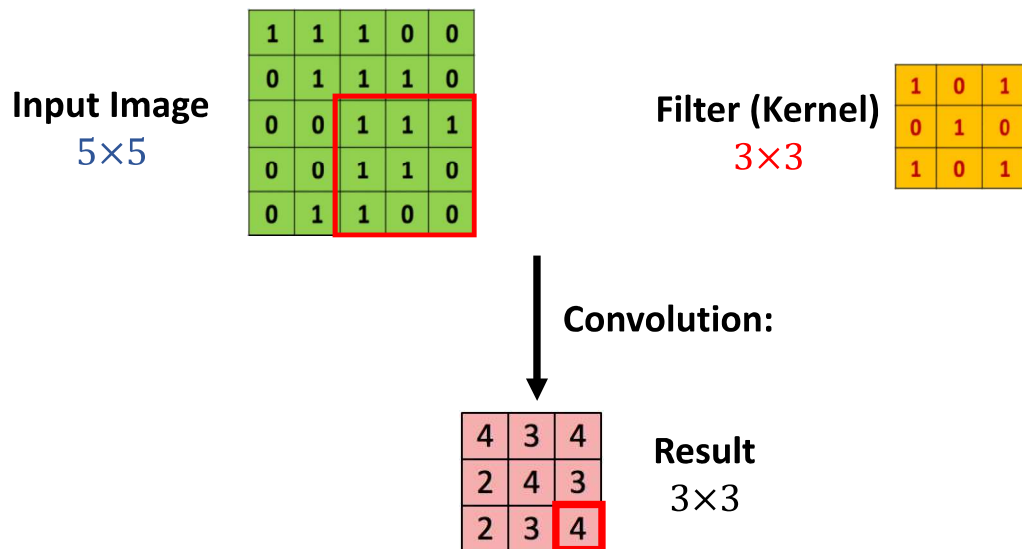
Convolution



Convolution



Convolution



Question: Why is the result a 3×3 matrix?

Convolution

Input Image
 4×4

1	1	1	0
0	1	1	1
0	1	1	0
1	1	0	0

Filter (Kernel)
 3×3

1	0	1
0	1	0
1	0	1

Convolution:

4	3
3	4

Result
 2×2

Convolution

Input Image
 5×5

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filter (Kernel)
 3×3

1	0	1
0	1	0
1	0	1

Convolution:

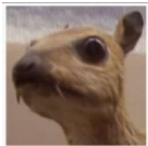
4	3	4
2	4	3
2	3	4

Dimensions:

- Input: $d_1 \times d_2$
- Filter: $k_1 \times k_2$
- Output: $(d_1 - k_1 + 1) \times (d_2 - k_2 + 1)$

Convolution

Input Image



Filters

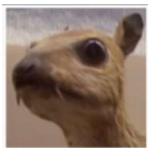
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$
----------	---

Outputs



Convolution

Input Image



Filters

Edge
detection

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

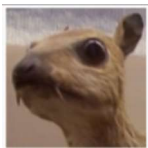
Outputs



- The inner product of $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$ and $\begin{bmatrix} 0.1 & 0.1 & 0.1 \\ 0.1 & 0.1 & 0.1 \\ 0.1 & 0.1 & 0.1 \end{bmatrix}$ is **0**.
- The inner product of $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$ and $\begin{bmatrix} 0.1 & 0.1 & 0.1 \\ 0.1 & 0.2 & 0.1 \\ 0.1 & 0.1 & 0.1 \end{bmatrix}$ is **0.8**.

Convolution

Input Image



Filters

Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

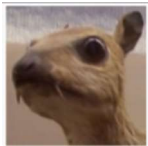
Outputs



- The inner product of $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ and $\begin{bmatrix} 0.1 & 0.1 & 0.1 \\ 0.1 & \mathbf{0.1} & 0.1 \\ 0.1 & 0.1 & 0.1 \end{bmatrix}$ is $\mathbf{0.1}$.
- The inner product of $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ and $\begin{bmatrix} 0.1 & 0.1 & 0.1 \\ 0.1 & \mathbf{0.2} & 0.1 \\ 0.1 & 0.1 & 0.1 \end{bmatrix}$ is $\mathbf{0.6}$.

Convolution

Input Image



Filters

Outputs
(or feature map)

Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Convolution



Input

Convolution: Zero Padding

Input Image
 5×5

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filter (Kernel)
 3×3

1	0	1
0	1	0
1	0	1

Convolution:

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Dimensions:

- Input: $d_1 \times d_2$
- Filter: 3×3
- Output: $(d_1 - 2) \times (d_2 - 2)$

Convolution: Zero Padding

1	1	1	0	0
0	0	0	0	0

--	--	--

Problem: the output is smaller than the input!

- If the input is 28×28 and the filter is 3×3 , you can have at most 13 convolutional layers. (The output of the 13th layer is 2×2 .)

Convolution:

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

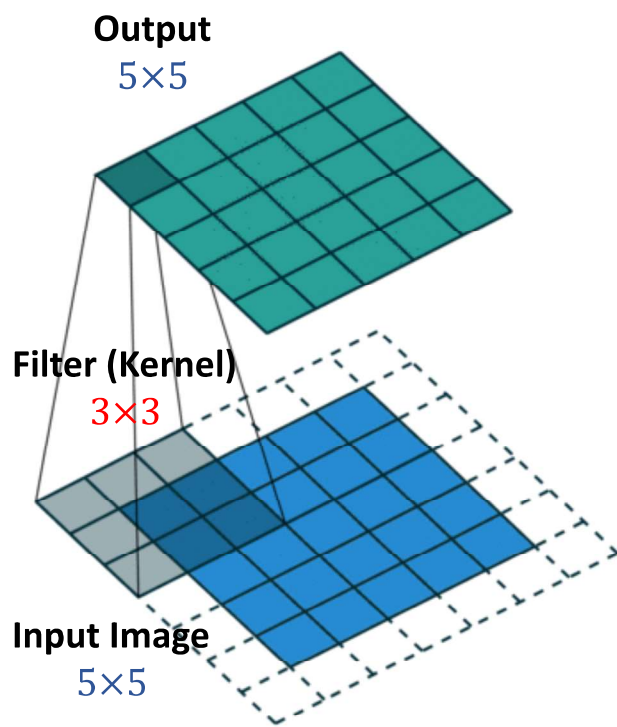
4		

Convolved
Feature

Dimensions:

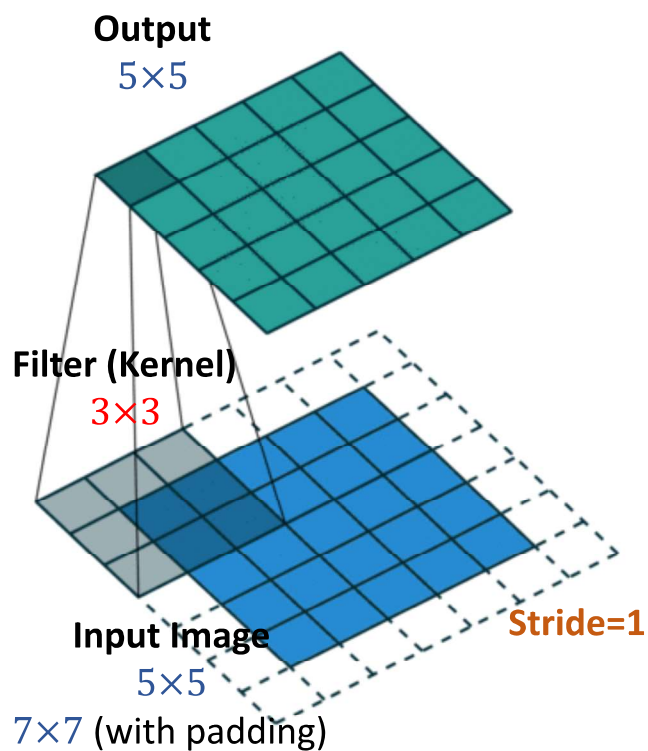
- Input: $d_1 \times d_2$
- Filter: 3×3
- Output: $(d_1 - 2) \times (d_2 - 2)$

Convolution: Zero Padding



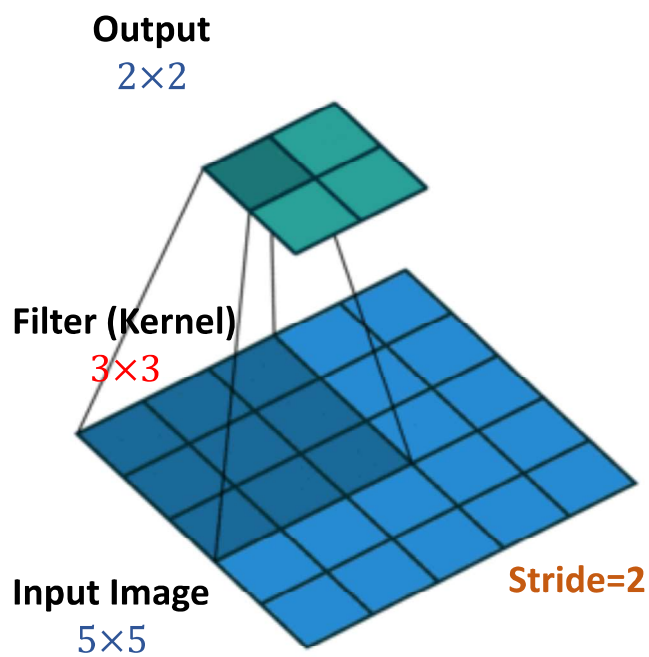
- Add a “border” of all-zeros.
- Increase the input shape:
 - From $d_1 \times d_2$ to $(d_1 + 2) \times (d_2 + 2)$.
- If the filter is 3×3 , the output is $d_1 \times d_2$.

Convolution: Stride



- In the previous examples, the **stride** is 1.
- The filter moves 1 step each time.

Convolution: Stride

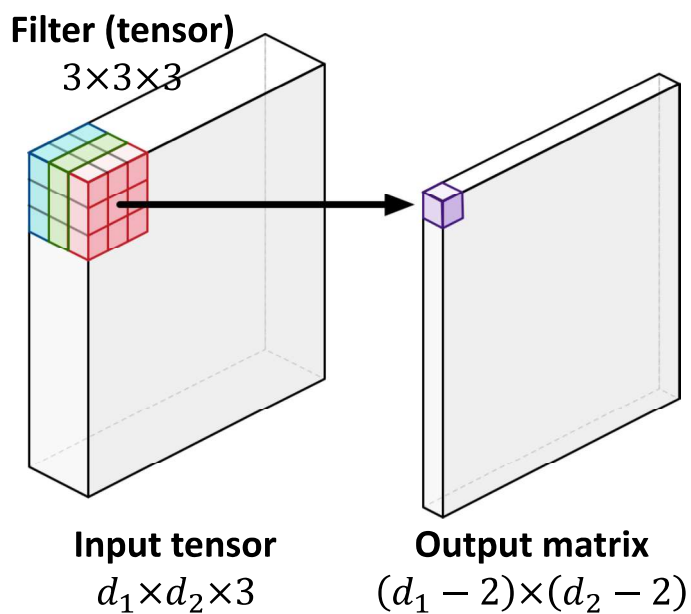


- In the previous examples, the **stride** is **1**.
 - The filter moves **1** step each time.
- The **stride** can be **2** or even larger.

Dimensions:

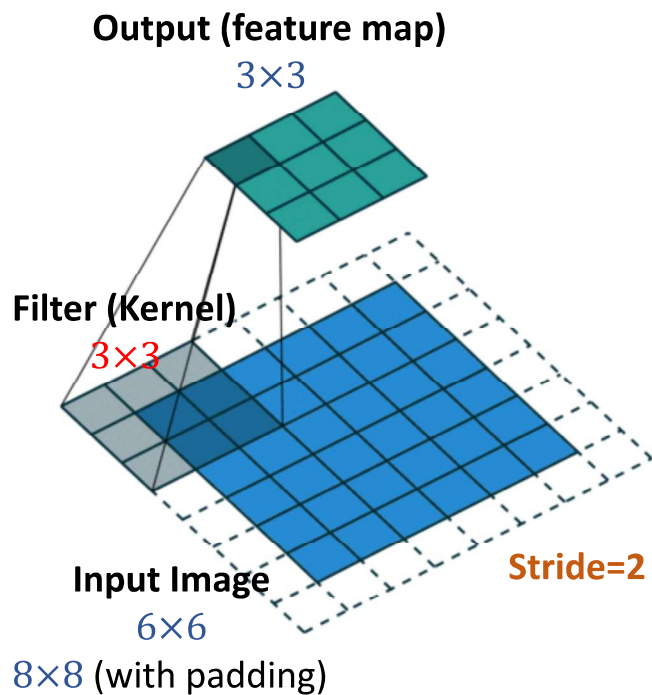
- Input: $d_1 \times d_2$
- Filter: $k_1 \times k_2$
- Stride: s
- Output: $\left(\left\lfloor \frac{d_1 - k_1}{s} \right\rfloor + 1\right) \times \left(\left\lfloor \frac{d_2 - k_2}{s} \right\rfloor + 1\right)$

Convolution for Tensors



- The input is the order-3 tensor of shape $d_1 \times d_2 \times 3$, e.g., a color image.
- The filter is a order-3 tensor of shape $3 \times 3 \times 3$.
- The output is a matrix (order-2 tensor) of shape $(d_1 - 2) \times (d_2 - 2)$.

Convolution: Key Concepts

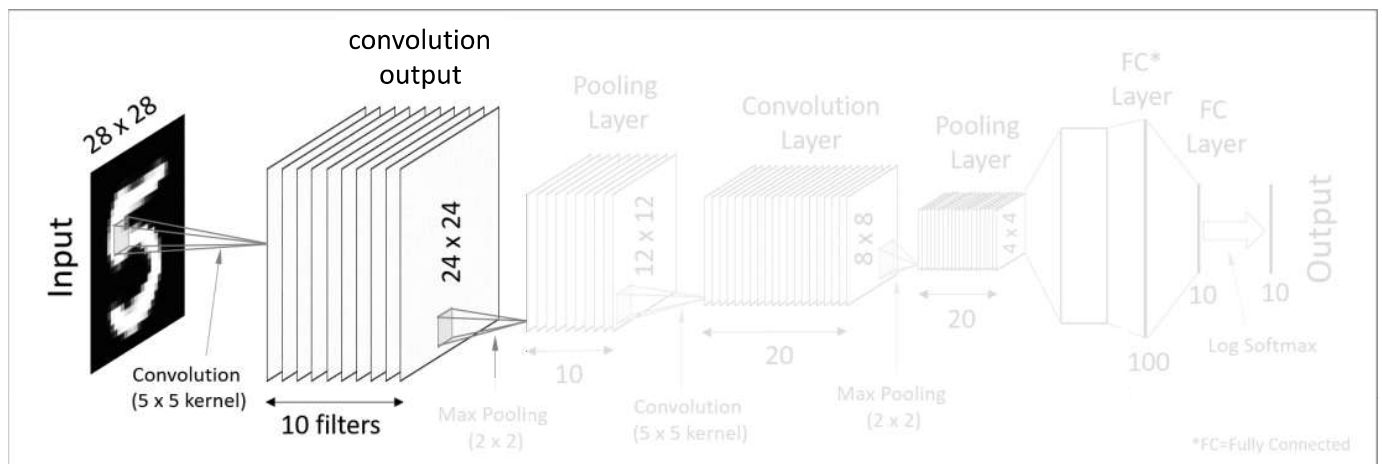


- Input, filter (kernel), and output (feature map).
- Convolution for matrix (order-2 tensor).
- Convolution for order-3 tensor.
- Zero padding.
- Stride.

Convolutional Neural Networks (CNNs)

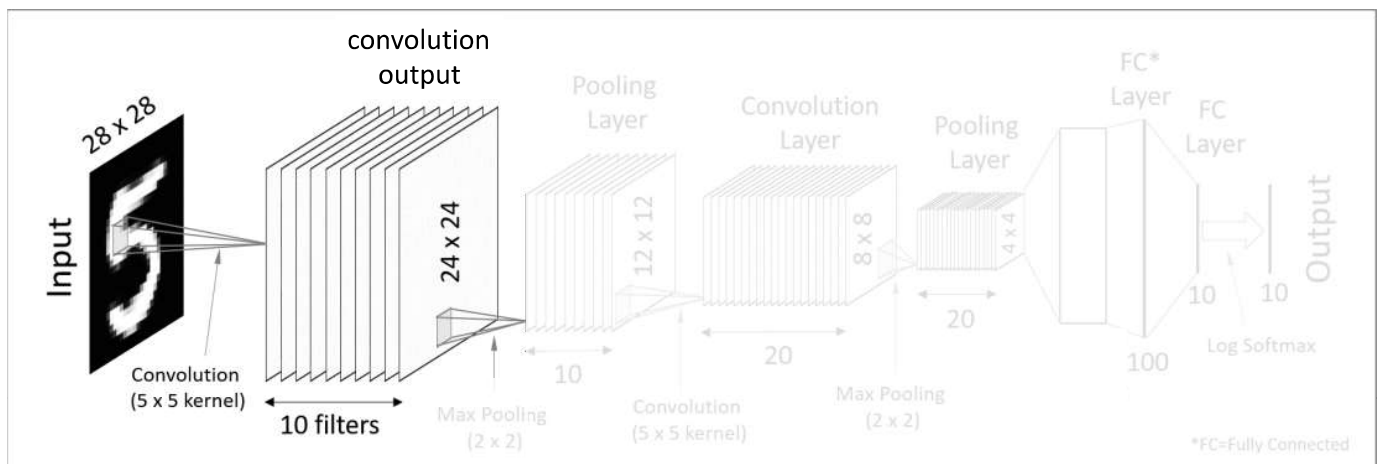
CNN: Convolutional Layer 1

- Use multiple filters (kernels) for the feature mapping. E.g.,
 - using ten 5×5 filters, (merely $10 \times 5 \times 5 = 250$ parameters!)
 - the results are ten 24×24 matrices.



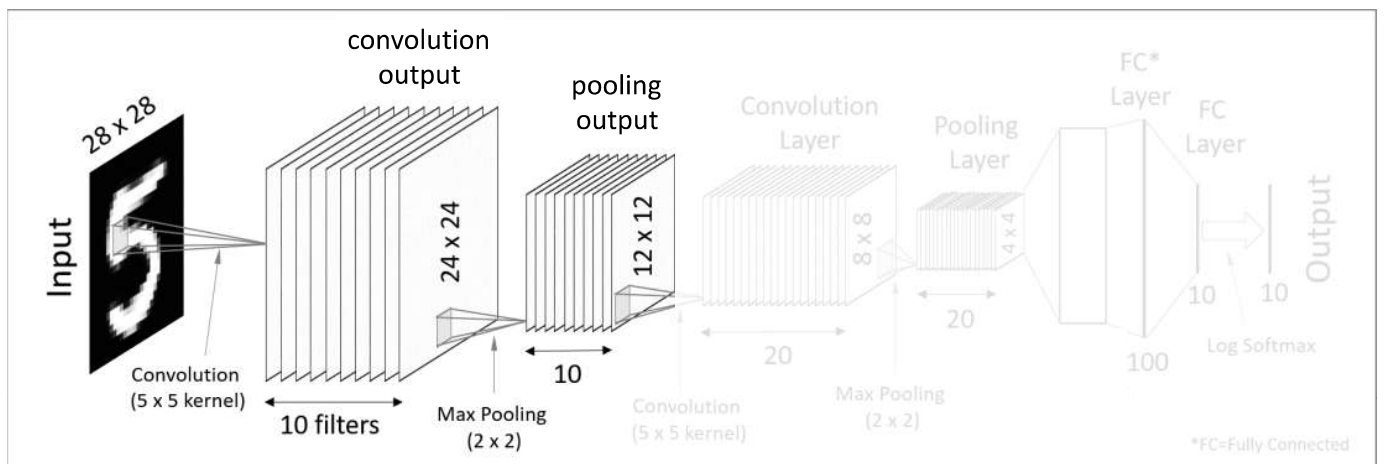
CNN: Convolutional Layer 1

- Use multiple filters (kernels) for the feature mapping.
- Apply an activation function (e.g., ReLU).
- A Convolutional layer = convolution + activation.



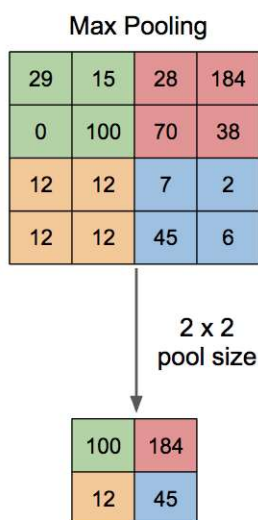
CNN: Pooling Layer 1

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.



Pooling

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.

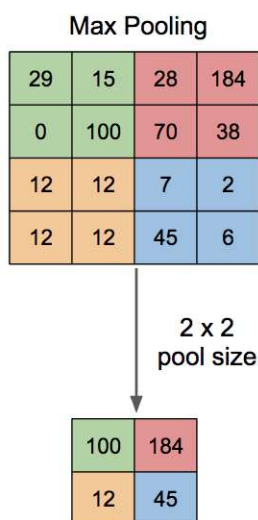


Settings:

- Max Pooling.
- Pool size = 2×2 .
- No overlap (stride = 2×2).

Pooling

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.



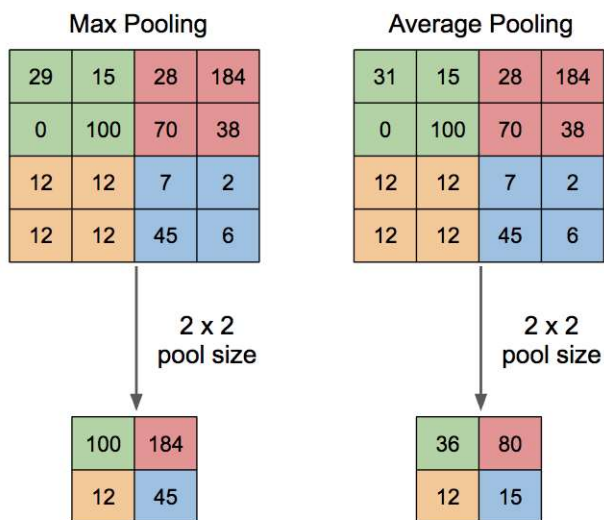
Settings:

- Max Pooling.
- Pool size = 2×2 .
- No overlap (stride = 2×2).

Question: What if the stride is 1×1 ?

Pooling

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.



Settings:

- Average Pooling.
- Pool size = 2×2 .
- No overlap (stride = 2×2).

Pooling

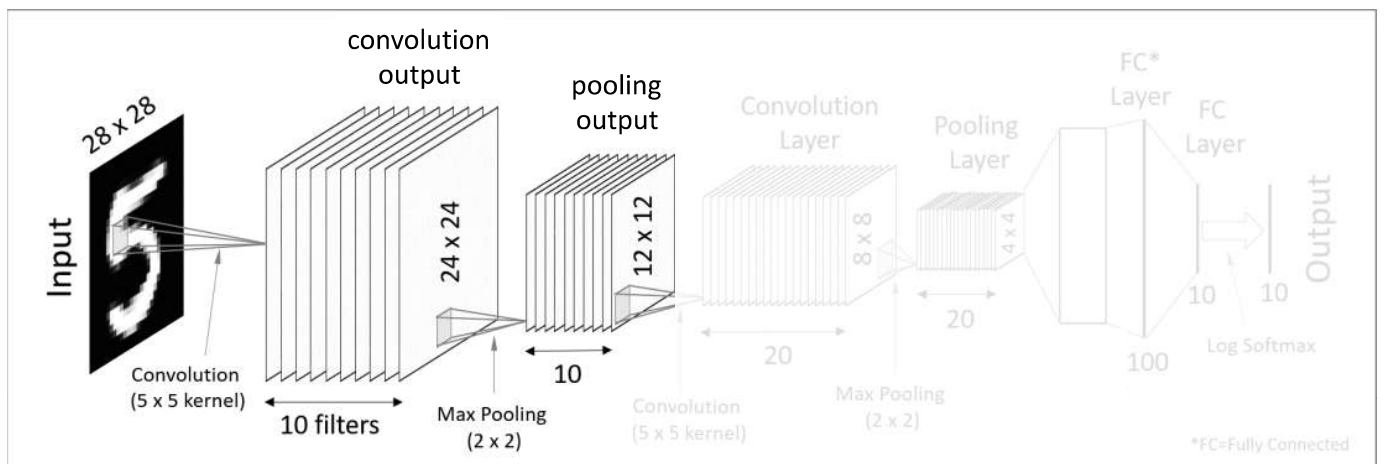
- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.
- **Question:** What is the output shape?
 - Input shape: 12×12 .
 - Pool size: 3×3 .
 - No overlap (stride: 3×3).

Pooling

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.
- **Question:** What is the output shape?
 - Input shape: 12×12 .
 - Pool size: 3×3 .
 - With overlap (stride: 1×1).

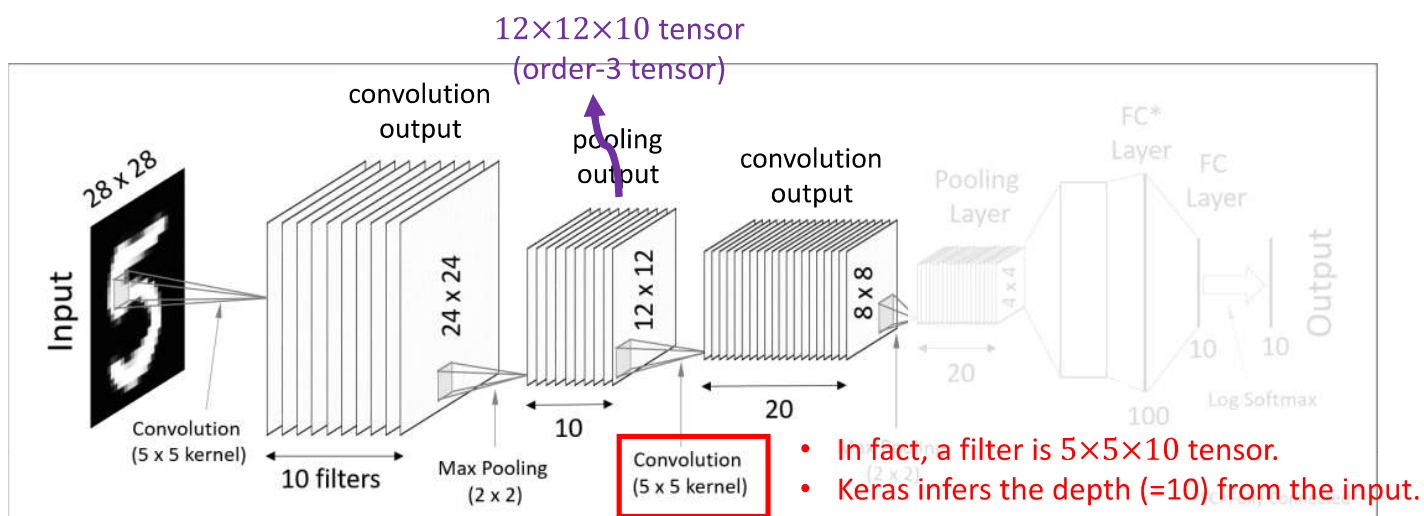
CNN: Pooling Layer 1

- Pooling reduces the dimensionality of each feature map.
- E.g., Max Pooling, Average Pooling, etc.

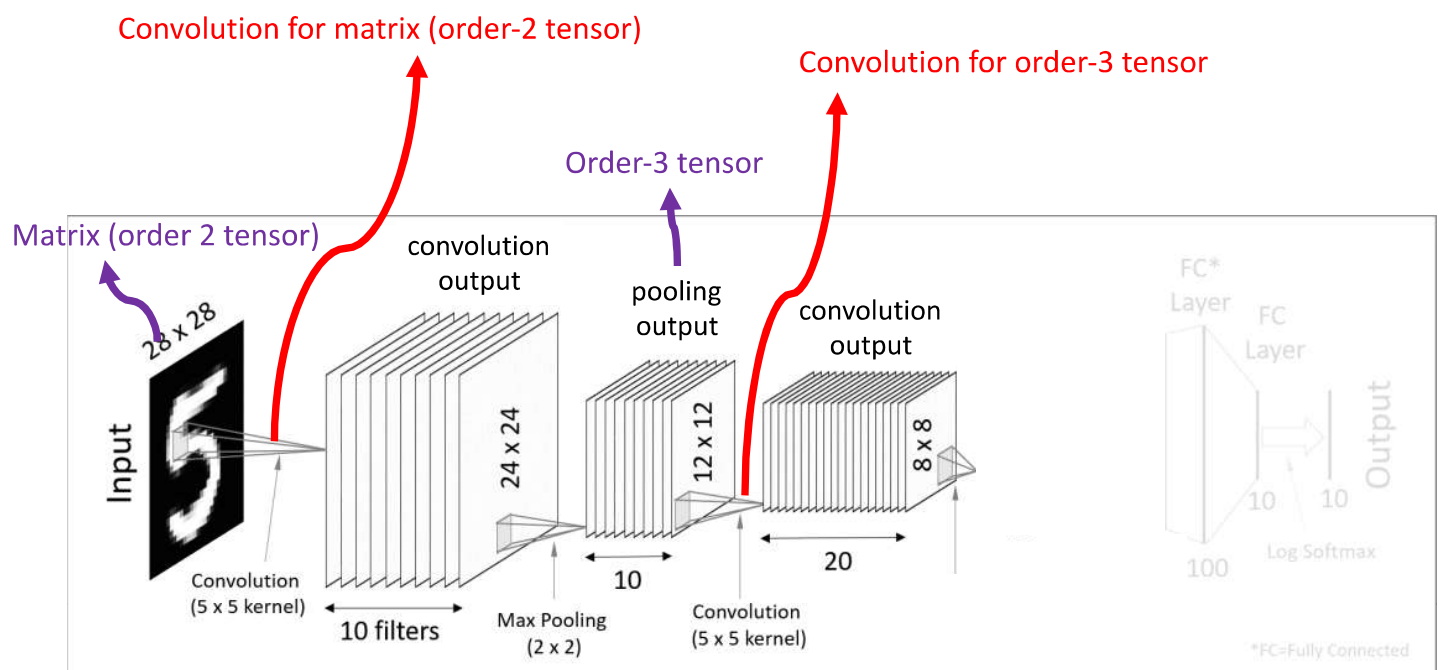


CNN: Convolutional Layer 2

- Perform tensor convolution.
- Apply $5 \times 5 \times 10$ tensor filter to the $12 \times 12 \times 10$ tensor input.
- Twenty such filters $\rightarrow$ twenty 8×8 feature maps.

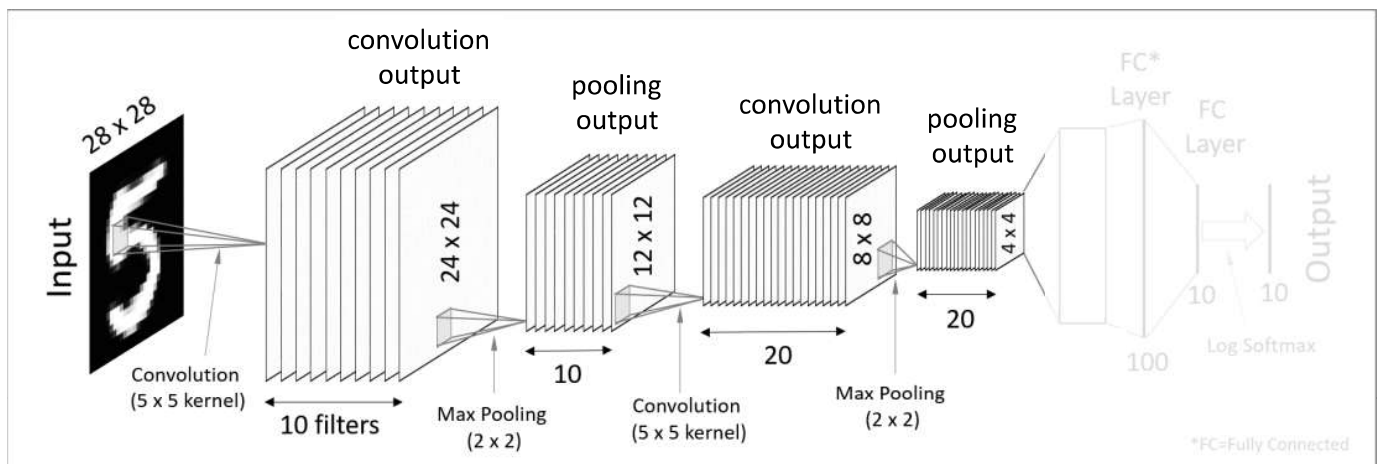


CNN: Convolutional Layers



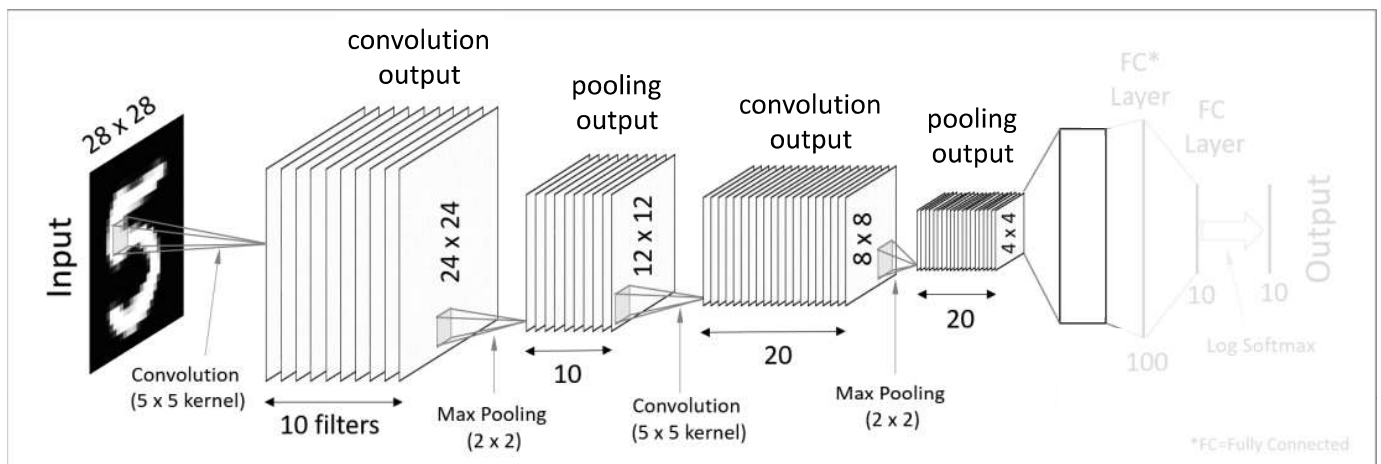
CNN: Pooling Layer 2

- 2×2 Max Pooling applied to an 8×8 matrix $\rightarrow$ a 4×4 matrix.



CNN: Flatten (Vectorization)

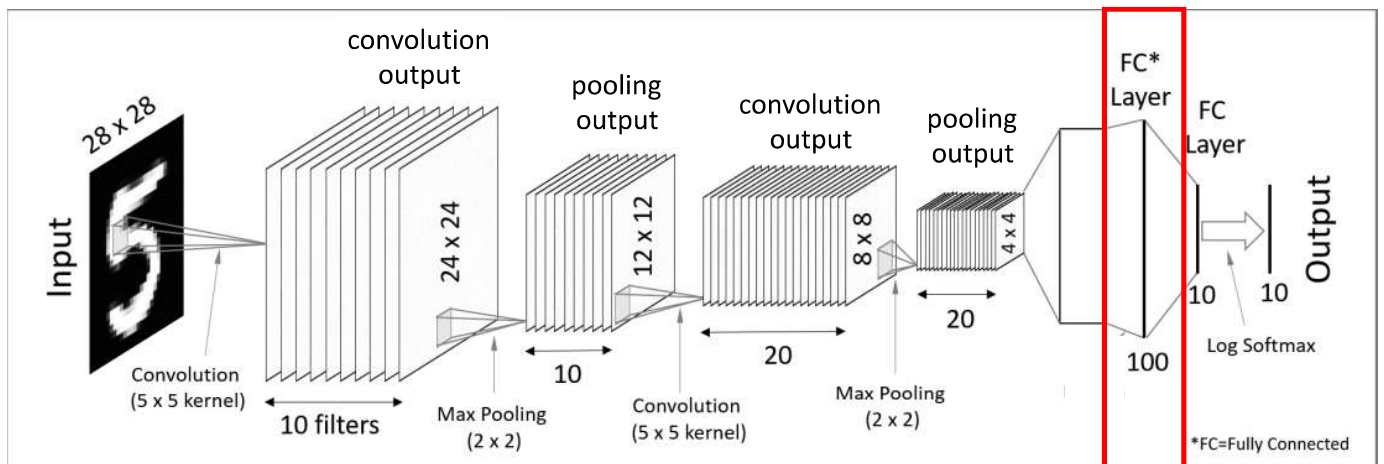
- Flatten the $4 \times 4 \times 20$ tensor.
- Get a 320-dim vector.



CNN: Fully-Connected Layers

- Add one or more fully-connected layers

This one is optional.



CNN: How Many Trainable Parameters?

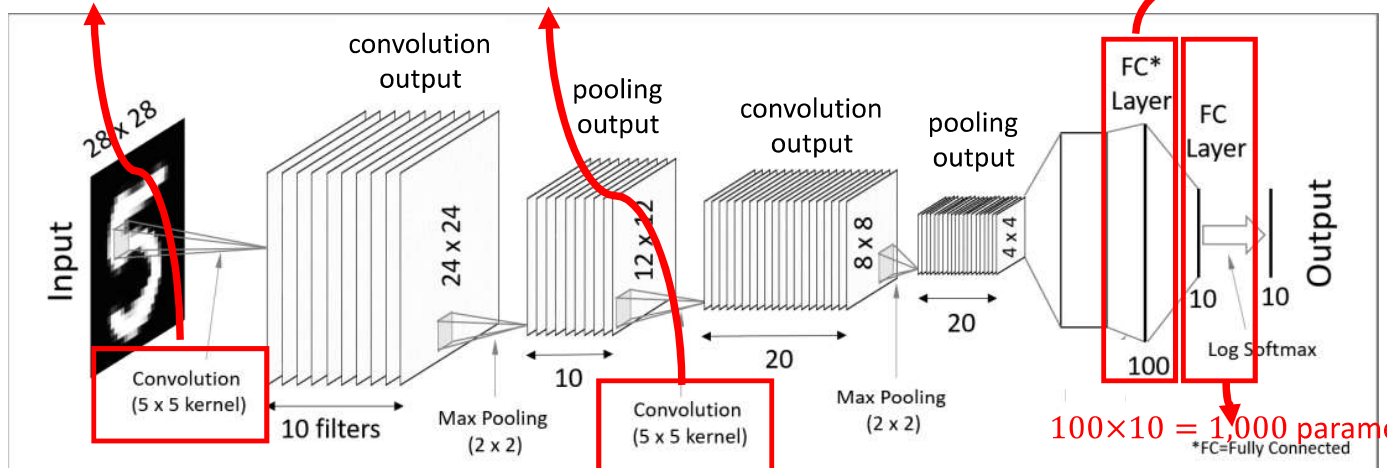
$$10 \times 5 \times 5 = 250 \text{ parameters}$$

- 10: number of filters
- 5×5: size of filters

$$20 \times 5 \times 5 \times 10 = 5000 \text{ parameters}$$

- 20: number of filters
- 5×5×10: size of filters

$$320 \times 100 = 32,000 \text{ parameters}$$



CNN: How Many Trainable Parameters?

Answer: $> 38,250$ parameters (not counting the intercepts).

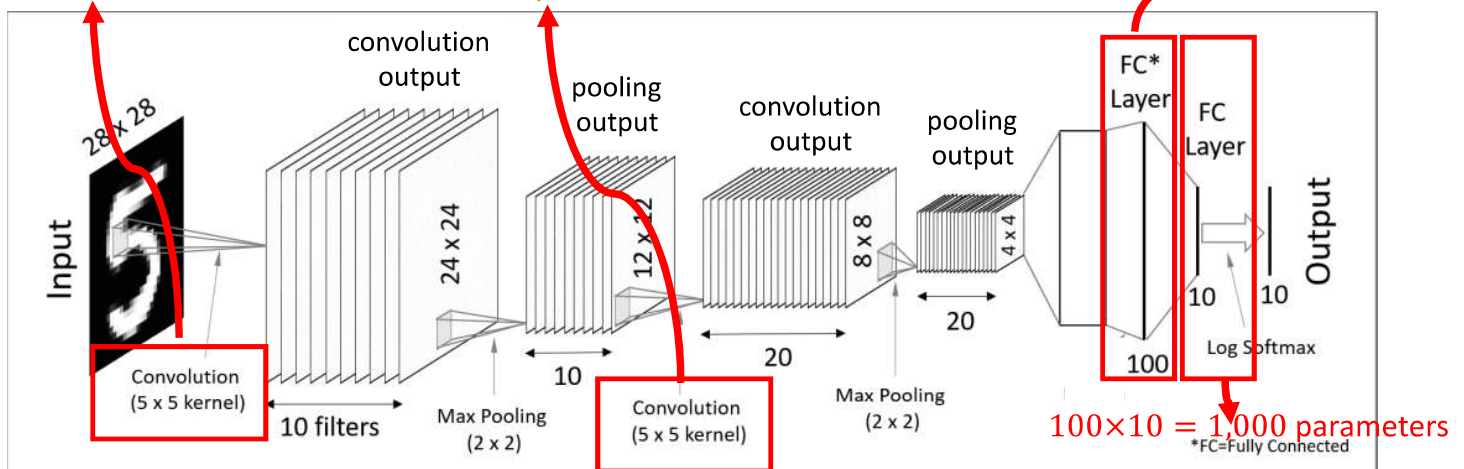
$$10 \times 5 \times 5 = 250 \text{ parameters}$$

- 10: number of filters
- 5×5: size of filters

$$20 \times 5 \times 5 \times 10 = 5000 \text{ parameters}$$

- 20: number of filters
- 5×5: size of filters
- 10: depth of filters

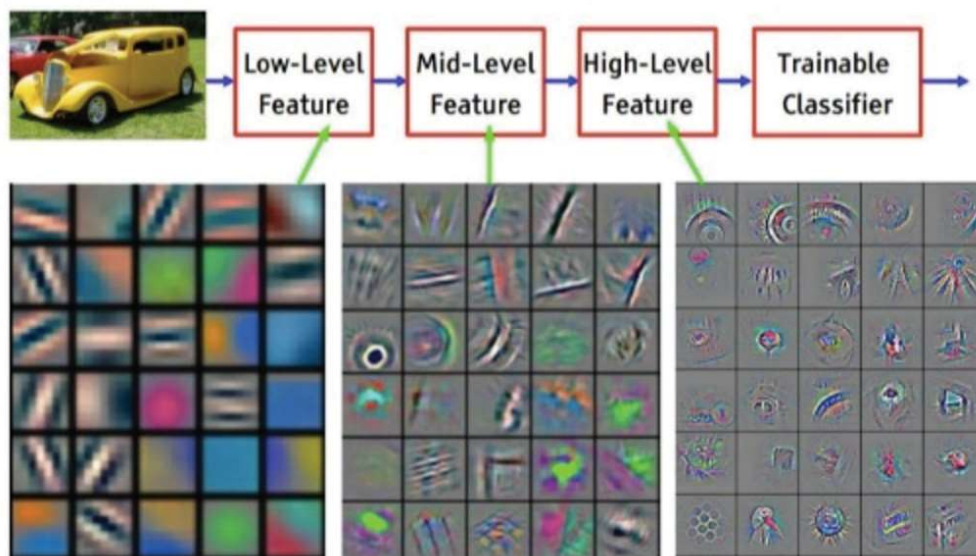
$$320 \times 100 = 32,000 \text{ parameters}$$



CNN: Re-Cap

Question: Why convolutional layers?

Answer: detect features like edge, shape, pattern, etc.



CNN: Re-Cap

Question: Why convolutional layers?

Answer: detect features like edge, shape, pattern, etc.

Question: Why activation functions like ReLU?

Answer: Add **nonlinearity** to increase the expressive power. (Note that convolution is a **linear** function.)

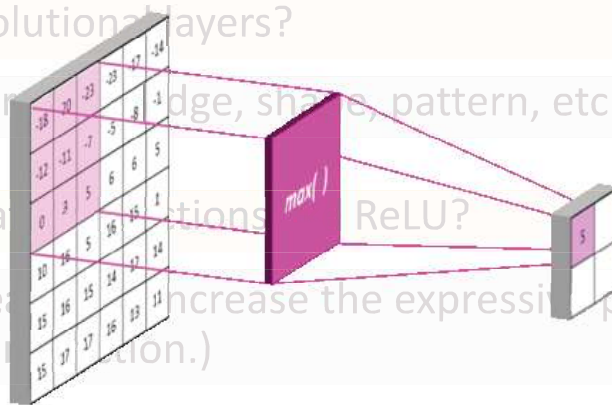
CNN: Re-Cap

Question: Why convolutional layers?

Answer: detect features like edge, shape, pattern, etc.

Question: Why activation functions like ReLU?

Answer: Add **nonlinearity** to increase the expressive power. (Note that convolution is a **linear** operation.)



Question: Why pooling?

Answer: Drastically reduce the shape of features.

Hyper-Parameters of CNN

Hyper-Parameters for Network Structure

- Convolutional layers

Filters

Filter shape

Stride

Zero-padding

- Pooling layers

MaxPool or AvgPool

Pool size

Pool stride

- Fully-connected layers

Width

- Activation functions

ReLU

SoftMax

Sigmoid

Hyper-Parameters for Training

- Loss function.

Cross-entropy for classification

L1 or L2 for regression (the labels are continuous)

- Optimization algorithm (and its hyper-parameters, e.g., learning rate).

SGD

SGD with momentum

AdaGrad

RMSprop

- Random initialization.

Gaussian or uniform?

Scaling

Implement a CNN Using Keras

1. Load and Process the MNIST Dataset

Load data

```
from keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

print('Shape of x_train: ' + str(x_train.shape))
print('Shape of x_test: ' + str(x_test.shape))
print('Shape of y_train: ' + str(y_train.shape))
print('Shape of y_test: ' + str(y_test.shape))
```

```
Shape of x_train: (60000, 28, 28)
Shape of x_test: (10000, 28, 28)
Shape of y_train: (60000,)
Shape of y_test: (10000,)
```

1. Load and Process the MNIST Dataset

Reshape: convert the $60000 \times 28 \times 28$ dataset to a $60000 \times 28 \times 28 \times 1$ tensor.

```
x_train_vec = x_train.reshape((60000, 28, 28, 1)) / 255.0
x_test_vec = x_test.reshape((10000, 28, 28, 1)) / 255.0

print('Shape of x_train_vec is ' + str(x_train_vec.shape))
```

Shape of x_train_vec is (60000, 28, 28, 1)

1. Load and Process the MNIST Dataset

One-hot encode: convert the **labels** (scalars in $\{0, 1, \dots, 9\}$) to 10-dim vectors.

```
def to_one_hot(labels, dimension=10):
    results = np.zeros((len(labels), dimension))
    for i, label in enumerate(labels):
        results[i, label] = 1.
    return results

y_train_vec = to_one_hot(y_train)
y_test_vec = to_one_hot(y_test)
print('Shape of y_train_vec is ' + str(y_train_vec.shape))
```

Shape of y_train_vec is (60000, 10)

1. Load and Process the MNIST Dataset

Partition the **training** set to **training** and **validation** sets.

```
# Partition to training and validation sets

rand_indices = np.random.permutation(60000)
train_indices = rand_indices[0:50000]
valid_indices = rand_indices[50000:60000]

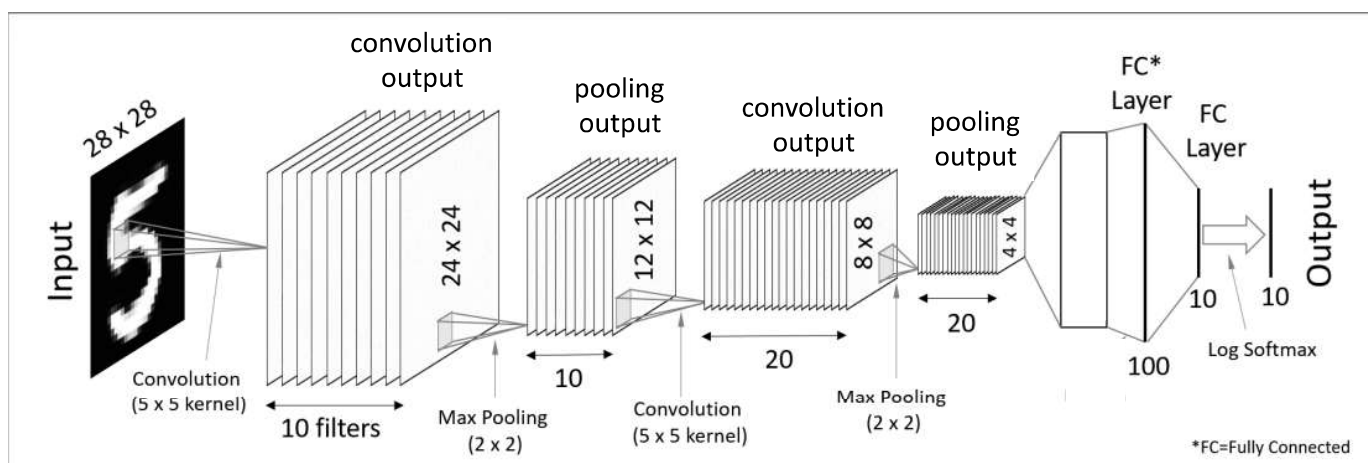
x_valid_vec = x_train_vec[valid_indices, :, :, :]
y_valid_vec = y_train_vec[valid_indices, :]

x_train_vec = x_train_vec[train_indices, :, :, :]
y_train_vec = y_train_vec[train_indices, :]

print('Shape of x_valid_vec: ' + str(x_valid_vec.shape))
print('Shape of y_valid_vec: ' + str(y_valid_vec.shape))
print('Shape of x_train_vec: ' + str(x_train_vec.shape))
print('Shape of y_train_vec: ' + str(y_train_vec.shape))

Shape of x_valid_vec: (10000, 28, 28, 1)
Shape of y_valid_vec: (10000, 10)
Shape of x_train_vec: (50000, 28, 28, 1)
Shape of y_train_vec: (50000, 10)
```

2. Build the CNN



2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
```



Filter number



Filter shape

Default

- Stride: 1
- Zero-padding: no padding

2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
```

pool_size

Default

- Stride: pool_size

2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2))) Output: 12×12×10
model.add(layers.Conv2D(20, (5, 5), activation='relu')) Output: 8×8×20
model.add(layers.MaxPooling2D((2, 2))) Output: 4×4×20
```

2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2))) Output: 12×12×10
model.add(layers.Conv2D(20, (5, 5), activation='relu')) Output: 8×8×20
model.add(layers.MaxPooling2D((2, 2))) Output: 4×4×20
model.add(layers.Flatten()) Output: 320
```

2. Build the CNN

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2))) Output: 12×12×10
model.add(layers.Conv2D(20, (5, 5), activation='relu')) Output: 8×8×20
model.add(layers.MaxPooling2D((2, 2))) Output: 4×4×20
model.add(layers.Flatten()) Output: 320
model.add(layers.Dense(100, activation='relu')) Output: 100
model.add(layers.Dense(10, activation='softmax')) Output: 10
```

2. Build the CNN

```
# print the summary of the model.  
model.summary()
```

Layer (type)	Output Shape	Param #
=====		
conv2d_1 (Conv2D)	(None, 24, 24, 10)	260

max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 10)	0

conv2d_2 (Conv2D)	(None, 8, 8, 20)	5020

max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 20)	0

flatten_1 (Flatten)	(None, 320)	0

dense_1 (Dense)	(None, 100)	32100

dense_2 (Dense)	(None, 10)	1010
=====		
Total params: 38,390		
Trainable params: 38,390		
Non-trainable params: 0		

3. Train the CNN

Specify: optimization algorithm, learning rate (LR), loss function, and metric.

```
from keras import optimizers
model.compile(optimizers.RMSprop(lr=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

3. Train the CNN

Specify: batch size and number of epochs.

```
history = model.fit(x_train_vec, y_train_vec,
                    batch_size=128, epochs=50,
                    validation_data=(x_valid_vec, y_valid_vec))
```

Train on 50000 samples, validate on 10000 samples

```
Epoch 1/50
50000/50000 [=====] - 12s 239us/step - loss: 1.2455 - acc: 0.7201 - val_loss: 0.4805 - val_acc: 0.8707
Epoch 2/50
50000/50000 [=====] - 12s 235us/step - loss: 0.3563 - acc: 0.8995 - val_loss: 0.2851 - val_acc: 0.9172
Epoch 3/50
50000/50000 [=====] - 12s 244us/step - loss: 0.2474 - acc: 0.9271 - val_loss: 0.2230 - val_acc: 0.9365
...
Epoch 49/50
50000/50000 [=====] - 12s 240us/step - loss: 0.0244 - acc: 0.9924 - val_loss: 0.0439 - val_acc: 0.9875
Epoch 50/50
50000/50000 [=====] - 12s 239us/step - loss: 0.0238 - acc: 0.9927 - val_loss: 0.0446 - val_acc: 0.9881
```

4. Examine the Results

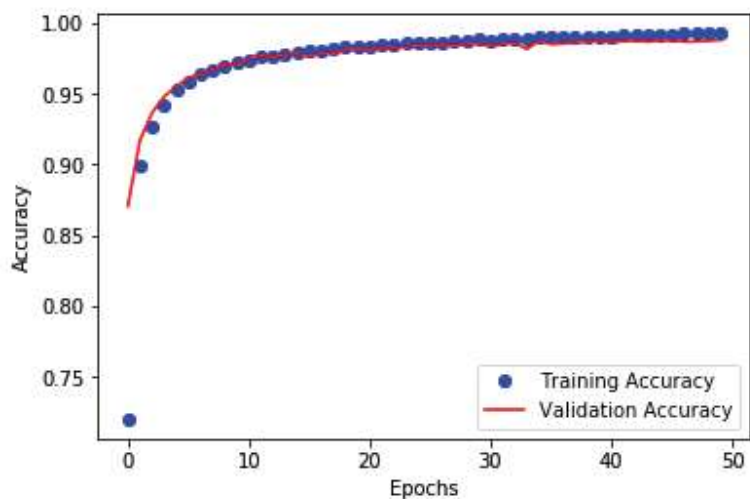
Plot the **accuracy** against **epochs** (1 epoch = 1 pass over the data).

```
import matplotlib.pyplot as plt
%matplotlib inline

epochs = range(50) # 50 is the number of epochs
train_acc = history.history['acc']
valid_acc = history.history['val_acc']
plt.plot(epochs, train_acc, 'bo', label='Training Accuracy')
plt.plot(epochs, valid_acc, 'r', label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

4. Examine the Results

Plot the **accuracy** against **epochs** (1 epoch = 1 pass over the data).



Why using the validation set?

- Tune the hyper-parameters, e.g., learning rate, filter shape, filter number, etc.
- Check overfitting or underfitting.

4. Examine the Results

Evaluate the model on the ***test*** set. (So far, the model has not seen the ***test*** set.)

```
loss_and_acc = model.evaluate(x_test_vec, y_test_vec)
print('loss = ' + str(loss_and_acc[0]))
print('accuracy = ' + str(loss_and_acc[1]))
```

```
10000/10000 [=====] - 1s 127us/step
loss = 0.03570800104729715
accuracy = 0.9883
```

4. Examine the Results

- Training accuracy: 99.3%
- Validation accuracy: 98.8%
- Test accuracy: 98.8%

Summary

1. Convolution

- Matrix convolutional and tensor convolution.
- Concepts:
 - filter (kernel)
 - patch
 - output (feature map)
 - zero-padding
 - stride

2. Convolutional Neural Network

- Convolutional layers.
 - Filter number.
 - Filter shape.
 - Stride (default: one).
 - Zero-padding (default: no padding).

2. Convolutional Neural Network

- Convolutional layers.
 - Filter number.
 - Filter shape.
 - Stride (default: one).
 - Zero-padding (default: no padding).
- Activation functions.
- Pooling layers.
 - MaxPool, AveragePool, etc.
 - Pool size.
 - Pool stride (default: pool size).

3. Build and Train CNN using Keras

- Build a network.
 - Add Conv, Pool, FC layers.
- Compile the model.
 - Specify optimization algorithm, loss function, and metrics.
- Fit the model on training data.

Batch Normalization

Feature Scaling for Linear Models

Why Feature Scaling?

People's feature vectors: $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^2$.

- The 1st dimension is one person's income (**in dollars**).
 - Assume it is randomly from the Gaussian distribution $N(3000, 400^2)$.
- The 2nd dimension is one person's height (**in inch**).
 - Assume it is randomly from the Gaussian distribution $N(69, 3^2)$.

Why Feature Scaling?

People's feature vectors: $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^2$.

- The 1st dimension is one person's income (**in dollars**).
 - Assume it is randomly from the Gaussian distribution $N(3000, 400^2)$.
- The 2nd dimension is one person's height (**in inch**).
 - Assume it is randomly from the Gaussian distribution $N(69, 3^2)$.

- Hessian matrix of least squares regression model:

$$\mathbf{H} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T = \begin{bmatrix} 9137.3 & 206.6 \\ 206.6 & 4.8 \end{bmatrix} \times 10^3.$$

- Condition number: $\frac{\lambda_{\max}(\mathbf{H})}{\lambda_{\min}(\mathbf{H})} = 9.2 \times 10^4$.

Bad condition number means slow convergence of gradient descent!

Why Feature Scaling?

People's feature vectors: $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^2$.

- The 1st dimension is one person's income (**in thousand dollars**).
 - Assume it is randomly from the Gaussian distribution $N(3, 0.4^2)$.
- The 2nd dimension is one person's height (**in foot**).
 - Assume it is randomly from the Gaussian distribution $N(5.75, 0.25^2)$.
- **Change metric.**
- Hessian matrix of linear regression:

$$\mathbf{H} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T = \begin{bmatrix} 9.1 & 17.2 \\ 17.2 & 33.1 \end{bmatrix}.$$

- Condition number: $\frac{\lambda_{\max}(\mathbf{H})}{\lambda_{\min}(\mathbf{H})} = 281.7$. (As opposed to 92K.)

Feature Scaling for **1D** Data

Assume the samples $x_1, \dots, x_n$ are **one-dimensional**.

- **Min-max normalization:** $x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$.
- After the scaling, the samples $x'_1, \dots, x'_n$ are in $[0, 1]$.

Feature Scaling for 1D Data

Assume the samples $x_1, \dots, x_n$ are **one-dimensional**.

- **Standardization:** $x'_i = \frac{x_i - \hat{\mu}}{\hat{\sigma}}$.
 - $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$ is the sample mean.
 - $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \hat{\mu})^2$ is the sample variance.
 - After the scaling, the samples $x'_1, \dots, x'_n$ have zero mean and unit variance.

Feature Scaling for **High-Dim** Data

- Independently perform feature scaling for every feature.
 - E.g., when scaling the “height” feature, ignore the “income” feature.

```
# Min-Max Normalization  
import numpy  
  
d = x.shape[1]  
xmin = numpy.min(x, axis=0).reshape(1, d)  
xmax = numpy.max(x, axis=0).reshape(1, d)  
xnew = (x - xmin) / (xmax - xmin)
```

Feature Scaling for **High-Dim** Data

- Independently perform feature scaling for every feature.
 - E.g., when scaling the “height” feature, ignore the “income” feature.

```
# Standardization  
import numpy  
  
d = x.shape[1]  
mu = numpy.mean(x, axis=0).reshape(1, d)  
sig = numpy.std(x, axis=0).reshape(1, d)  
xnew = (x - mu) / sig
```

Batch Normalization

Batch Normalization: Standardization of Hidden Layers

- Let $\mathbf{x}^{(k)} \in \mathbb{R}^d$ be the output of the k -th hidden layer.
- $\hat{\boldsymbol{\mu}} \in \mathbb{R}^d$: sample mean of $\mathbf{x}^{(k)}$ evaluated on a batch of samples.
- $\hat{\boldsymbol{\sigma}} \in \mathbb{R}^d$: sample std of $\mathbf{x}^{(k)}$ evaluated on a batch of samples.

- Standardization: $z_j^{(k)} = \frac{x_j^{(k)} - \hat{\mu}_j}{\hat{\sigma}_j + 0.001}$, for $j = 1, \dots, d$.

Batch Normalization: Standardization of Hidden Layers

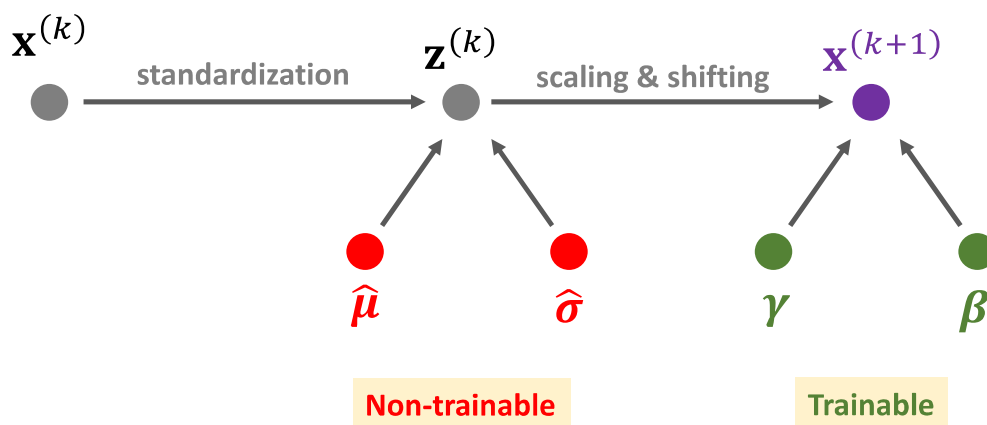
- Let $\mathbf{x}^{(k)} \in \mathbb{R}^d$ be the output of the k -th hidden layer.
- $\hat{\boldsymbol{\mu}} \in \mathbb{R}^d$: sample mean of $\mathbf{x}^{(k)}$ evaluated on a batch of samples.
- $\hat{\boldsymbol{\sigma}} \in \mathbb{R}^d$: sample std of $\mathbf{x}^{(k)}$ evaluated on a batch of samples.
- $\boldsymbol{\gamma} \in \mathbb{R}^d$: scaling parameter (trainable).
- $\boldsymbol{\beta} \in \mathbb{R}^d$: shifting parameter (trainable).
- Standardization: $z_j^{(k)} = \frac{x_j^{(k)} - \hat{\mu}_j}{\hat{\sigma}_j + 0.001}$, for $j = 1, \dots, d$.
- Scale and shift: $x_j^{(k+1)} = z_j^{(k)} \cdot \gamma_j + \beta_j$, for $j = 1, \dots, d$.

Batch Normalization: Standardization of Hidden Layers

- Let $\mathbf{x}^{(k)} \in \mathbb{R}^d$ be the output of the k -th hidden layer.
- $\hat{\boldsymbol{\mu}} \in \mathbb{R}^d$: **Non-trainable**. Just record them in the forward pass;
- $\hat{\boldsymbol{\sigma}} \in \mathbb{R}^d$: use them in the backpropagation.
- $\boldsymbol{\gamma} \in \mathbb{R}^d$: scaling parameter (**trainable**).
- $\boldsymbol{\beta} \in \mathbb{R}^d$: shifting parameter (**trainable**).
- Standardization: $z_j^{(k)} = \frac{x_j^{(k)} - \hat{\mu}_j}{\hat{\sigma}_j + 0.001}$, for $j = 1, \dots, d$.
- Scale and shift: $x_j^{(k+1)} = z_j^{(k)} \cdot \gamma_j + \beta_j$, for $j = 1, \dots, d$.

Backpropagation for Batch Normalization Layer

- Standardization: $z_j^{(k)} = \frac{x_j^{(k)} - \hat{\mu}_j}{\hat{\sigma}_j + 0.001}$, for $j = 1, \dots, d$.
- Scale and shift: $x_j^{(k+1)} = z_j^{(k)} \circ \gamma_j + \beta_j$, for $j = 1, \dots, d$.



Backpropagation for Batch Normalization Layer

- Standardization: $z_j^{(k)} = \frac{x_j^{(k)} - \hat{\mu}_j}{\hat{\sigma}_j + 0.001}$, for $j = 1, \dots, d$.
- Scale and shift: $x_j^{(k+1)} = z_j^{(k)} \circ \gamma_j + \beta_j$, for $j = 1, \dots, d$.

We know $\frac{\partial L}{\partial x_j^{(k+1)}}$ from the backpropagation (from the top to $x^{(k+1)}$.)

- Use $\frac{\partial L}{\partial \gamma_j} = \frac{\partial L}{\partial x_j^{(k+1)}} \frac{\partial x_j^{(k+1)}}{\partial \gamma_j} = \frac{\partial L}{\partial x_j^{(k+1)}} z_j^{(k)}$ to update γ_j ;
- Use $\frac{\partial L}{\partial \beta_j} = \frac{\partial L}{\partial x_j^{(k+1)}} \frac{\partial x_j^{(k+1)}}{\partial \beta_j} = \frac{\partial L}{\partial x_j^{(k+1)}}$ to update β_j .

Backpropagation for Batch Normalization Layer

- Standardization: $z_j^{(k)} = \frac{x_j^{(k)} - \hat{\mu}_j}{\hat{\sigma}_j + 0.001}$, for $j = 1, \dots, d$.
- Scale and shift: $x_j^{(k+1)} = z_j^{(k)} \circ \gamma_j + \beta_j$, for $j = 1, \dots, d$.

We know $\frac{\partial L}{\partial x_j^{(k+1)}}$ from the backpropagation (from the top to $x^{(k+1)}$.)

Compute $\frac{\partial L}{\partial z_j^{(k)}} = \frac{\partial L}{\partial x_j^{(k+1)}} \frac{\partial x_j^{(k+1)}}{\partial z_j^{(k)}} = \frac{\partial L}{\partial x^{(k+1)}} \gamma_j$.

Compute $\frac{\partial L}{\partial x_j^{(k)}} = \frac{\partial L}{\partial z_j^{(k)}} \frac{\partial z_j^{(k)}}{\partial x_j^{(k)}} = \frac{\partial L}{\partial z_j^{(k)}} \frac{1}{\hat{\sigma}_j + 0.001}$ and pass it to the lower layers.

Batch Normalization Layer in Keras

Batch Normalization Layer

- Let $\mathbf{x}^{(k)} \in \mathbb{R}^d$ be the output of the k -th hidden layer.
- $\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\sigma}} \in \mathbb{R}^d$: **non-trainable** parameters.
- $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$: **trainable** parameters.
- Standardization: $z_j^{(k)} = \frac{x_j^{(k)} - \hat{\mu}_j}{\hat{\sigma}_j + 0.001}$, for $j = 1, \dots, d$.
- Scale and shift: $x_j^{(k+1)} = z_j^{(k)} \cdot \gamma_j + \beta_j$, for $j = 1, \dots, d$.

Batch Normalization Layer

- Let $\mathbf{x}^{(k)} \in \mathbb{R}^d$ be the output of the k -th hidden layer.
- $\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\sigma}} \in \mathbb{R}^d$: **non-trainable** parameters.
- $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$: **trainable** parameters.

Difficulty: There are $4d$ parameters which must be stored in memory.
 d can be very large!

Example:

- The 1st Conv Layer in VGG16 Net outputs a $150 \times 150 \times 64$ tensor.
- The number of parameters in a single Batch Normalization Layer would be $4d = 1.44\text{M}$.

Batch Normalization Layer

Solution:

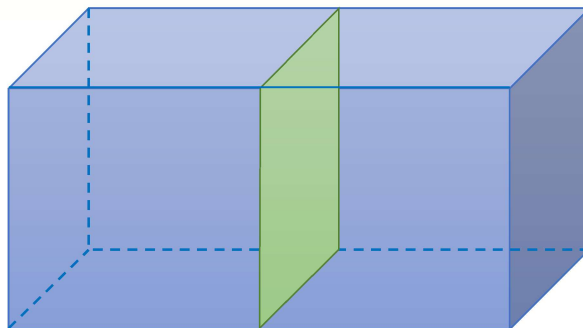
- Make the 4 parameters $1 \times 1 \times 64$, instead of $150 \times 150 \times 64$.

α can be very large:

Batch Normalization Layer

Solution:

- Make the 4 parameters $1 \times 1 \times 64$, instead of $150 \times 150 \times 64$.
- How?
- A scalar parameter for a slice (e.g., a 150×150 matrix) of the tensor.
- Of course, you can make the parameters $150 \times 1 \times 1$ or $1 \times 150 \times 1$.



CNN for Digit Recognition

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), input_shape=(28, 28, 1)))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(20, (5, 5)))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(100))
model.add(layers.BatchNormalization())
model.add(layers.Activation('relu'))
model.add(layers.Dense(10, activation='softmax'))
```

CNN for Digit Recognition

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 24, 24, 10)	260
batch_normalization_1 (Batch Normalization)	(None, 24, 24, 10)	40
activation_1 (Activation)	(None, 24, 24, 10)	0
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 10)	0
conv2d_2 (Conv2D)	(None, 8, 8, 20)	5020
batch_normalization_2 (Batch Normalization)	(None, 8, 8, 20)	80
activation_2 (Activation)	(None, 8, 8, 20)	0
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 20)	0
flatten_1 (Flatten)	(None, 320)	0
dense_1 (Dense)	(None, 100)	32100
batch_normalization_3 (Batch Normalization)	(None, 100)	400
activation_3 (Activation)	(None, 100)	0
dense_2 (Dense)	(None, 10)	1010
Total params: 38,910		
Trainable params: 38,650		
Non-trainable params: 260		

CNN for Digit Recognition

Train the model (**with** Batch Normalization) on MNIST ($n = 50,000$).

Train on 50000 samples, validate on 10000 samples

Epoch 1/3

50000/50000 [=====] - 29s 580us/step -
loss: 0.1599 - acc: 0.9595 - val_loss: 0.1165 - val_acc: 0.9644

Epoch 2/3

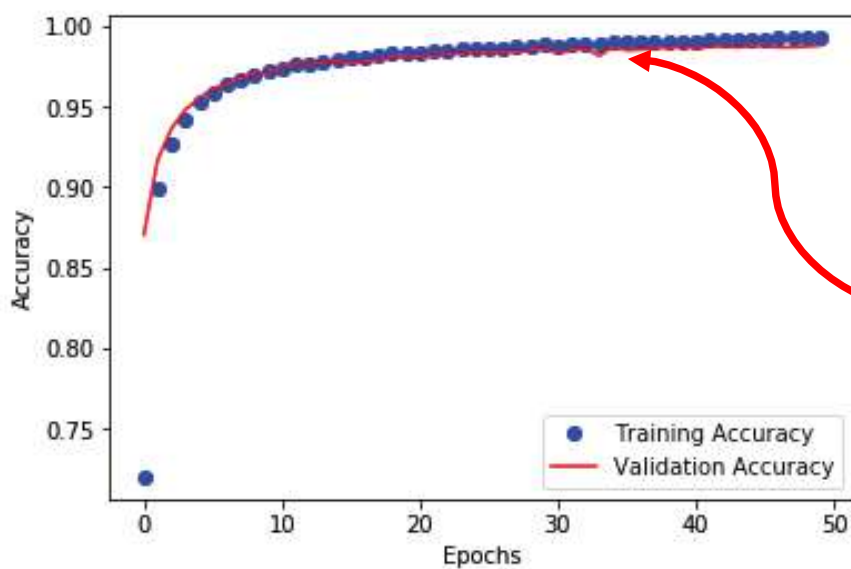
50000/50000 [=====] - 26s 516us/step -
loss: 0.0468 - acc: 0.9858 - val_loss: 0.0562 - val_acc: 0.9822

Epoch 3/3

50000/50000 [=====] - 25s 508us/step -
loss: 0.0325 - acc: 0.9902 - val_loss: 0.0494 - val_acc: 0.9832

CNN for Digit Recognition

Train the model (**without** Batch Normalization) on MNIST ($n = 50,000$).



Without Batch Normalization, it takes 10x more epochs to converge.

Summary

Feature Scaling

- Make the scales of all the features comparable.
- Why? Better conditioned Hessian matrix → Faster convergence.

Feature Scaling

- Make the scales of all the features comparable.
- Why? Better conditioned Hessian matrix → Faster convergence.
- Methods:
 - Min-Max Normalization: scale the features to $[0, 1]$.
 - Standardization: every feature has zero mean and unit variance.

Batch Normalization

- Feature standardization for the hidden layers.
- Why? Faster convergence.
- 2 trainable parameters: **shifting and scaling**.
- 2 non-trainable parameters: **mean, variance**.

Batch Normalization

- Feature standardization for the hidden layers.
- Why? Faster convergence.
- 2 trainable parameters: `shifting and scaling`.
- 2 non-trainable parameters: `mean, variance`.
- Keras provides `layers.BatchNormalization()`.
- Put the BN layer `after Conv` and `before Activation`.

Optimization for deep learning

(1) Optimization and deep learning

Gradient descent, stochastic gradient descent and mini-batch SGD

(2) Gradient descent optimization algorithms

Momentum, Nesterov accelerated gradient, Adagrad, RMSProp, Adam, AMSGrad

(3) PyTorch optimizers

(1) Some warnings about optimization in deep learning?

The objective function of an optimization algorithm is usually a loss function based on the training data set, hence the goal of optimization is to reduce the [training error](#).

(1) Some warnings about optimization in deep learning?

The objective function of an optimization algorithm is usually a loss function based on the training data set, hence the goal of optimization is to reduce the [training error](#).

However, the goal of (deep) learning is to reduce the [generalization error](#).

(1) Some warnings about optimization in deep learning?

The objective function of an optimization algorithm is usually a loss function based on the training data set, hence the goal of optimization is to reduce the [training error](#).

However, the goal of (deep) learning is to reduce the [generalization error](#).

In order to reduce the generalization error, we need to pay attention to [overfitting](#) in addition to using the optimization algorithm to reduce the training error.

(1) Some warnings about optimization in deep learning?

The objective function of an optimization algorithm is usually a loss function based on the training data set, hence the goal of optimization is to reduce the [training error](#).

However, the goal of (deep) learning is to reduce the [generalization error](#).

In order to reduce the generalization error, we need to pay attention to [overfitting](#) in addition to using the optimization algorithm to reduce the training error.

In this course, we focus specifically on the [performance](#) of the optimization algorithm in minimizing the objective function, rather than the model's generalization error.

In the next lessons, we will see techniques to avoid [overfitting](#).

(1) Some warnings about optimization in deep learning?

The objective function of an optimization algorithm is usually a loss function based on the training data set, hence the goal of optimization is to reduce the [training error](#).

However, the goal of (deep) learning is to reduce the [generalization error](#).

In order to reduce the generalization error, we need to pay attention to [overfitting](#) in addition to using the optimization algorithm to reduce the training error.

In this course, we focus specifically on the [performance](#) of the optimization algorithm in minimizing the objective function, rather than the model's generalization error.

In the next lessons, we will see techniques to avoid [overfitting](#).

No theorem in this lecture (for theorems, see a convex optimization course). We focus on intuitions because deep learning optimization problems are not convex and we hope our [intuition](#) built on convex problems will be useful!

(1) What do we optimize?

Recall the simple linear regression where the goal is to minimize the **cost function**:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (y(i) - \theta^T x(i))^2.$$

(1) What do we optimize?

Recall the simple linear regression where the goal is to minimize the **cost function**:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (y(i) - \theta^T x(i))^2.$$

For a deep neural network $F(\cdot; \theta)$ with parameters θ , we minimize the objective function:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (y(i) - F(x(i); \theta))^2,$$

in the parameter θ on the **training set**.

This problem is not convex anymore but we can still compute the gradient of the cost function $\nabla J(\theta)$ with respect to the parameters (well PyTorch do it for us!).

(1) Gradient descent variants

Idea: update the parameters in the opposite direction of the gradient.

The **learning rate** η determines the size of the steps.

(1) Gradient descent variants

Idea: update the parameters in the opposite direction of the gradient.

The **learning rate** η determines the size of the steps.

Batch gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

(1) Gradient descent variants

Idea: update the parameters in the opposite direction of the gradient.

The **learning rate** η determines the size of the steps.

Batch gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

Stochastic gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t; x(i), y(i))$$

(1) Gradient descent variants

Idea: update the parameters in the opposite direction of the gradient.

The **learning rate** η determines the size of the steps.

Batch gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

Stochastic gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t; x(i), y(i))$$

Mini-batch gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t; x(i : i + b), y(i : i + b)),$$

where b is the batch size.

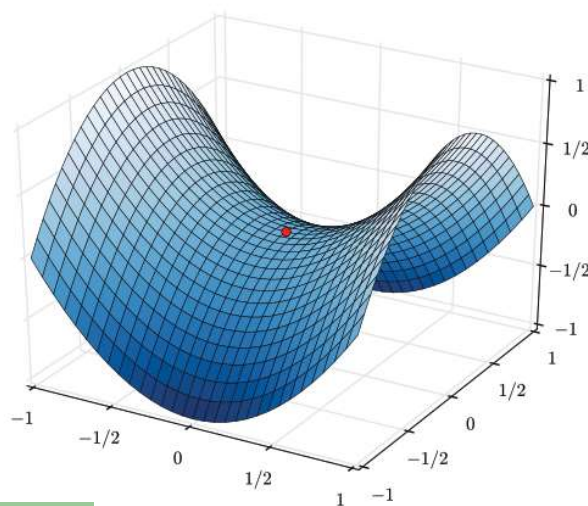
(1) Challenges

Mini-batch gradient descent is the algorithm of choice when training a neural network. The term SGD is usually employed also when mini-batches are used!

(1) Challenges

Mini-batch gradient descent is the algorithm of choice when training a neural network. The term SGD is usually employed also when mini-batches are used!

- Choosing a proper learning rate can be difficult. How to adapt the learning rate during training?
- Why applying the same learning rate to all parameters updates?
- How to escape saddle points where the gradient is close to zero in all dimension?



(1) Challenges

Mini-batch gradient descent is the algorithm of choice when training a neural network. The term SGD is usually employed also when mini-batches are used!

- Choosing a proper learning rate can be difficult. How to adapt the learning rate during training?
- Why applying the same learning rate to all parameters updates?
- How to escape saddle points where the gradient is close to zero in all dimension?

In the rest of the lecture, we will introduce [modifications to SGD](#).

Also we will apply these modifications to mini-batch gradient descent, we omit the explicit reference to the batch $x(i : i + b), y(i : i + b)$. Hence we write for SGD:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t),$$

but we need to keep in mind that updates are performed for every mini-batch of b training samples.

(2) Gradient descent optimization algorithms

A nice [survey](#) by Sebastian Ruder

Please run the [jupyter notebook](#) in parallel as you will need to code the optimization algorithms as soon as we see them.

We will deal with the toy problem: minimize in x_1, x_2 , the function:

$$f(x_1, x_2) = 0.1x_1^2 + 2x_2^2.$$

(2) Gradient descent optimization algorithms

A nice [survey](#) by Sebastian Ruder

Please run the [jupyter notebook](#) in parallel as you will need to code the optimization algorithms as soon as we see them.

We will deal with the toy problem: minimize in x_1, x_2 , the function:

$$f(x_1, x_2) = 0.1x_1^2 + 2x_2^2.$$

You will need to implement all variations of SGD.

Have a look at the `train_2d()` function and the Gradient descent code to understand what this function does.

(2) Momentum

Accelerating SGD by dampening oscillations, i.e. by averaging the last values of the gradient.

(2) Momentum

Accelerating SGD by dampening oscillations, i.e. by averaging the last values of the gradient.

$$v_{t+1} = \gamma v_t + \eta \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

(2) Momentum

Accelerating SGD by dampening oscillations, i.e. by averaging the last values of the gradient.

$$\begin{aligned}v_{t+1} &= \gamma v_t + \eta \nabla J(\theta_t) \\ \theta_{t+1} &= \theta_t - v_{t+1}\end{aligned}$$

Why does it work?

With $g_t = \nabla J(\theta_t)$, we have for any $k > 0$:

$$v_{t+1} = \gamma^k v_{t-k} + \eta \underbrace{\sum_{i=0}^k \gamma^i g_{t-i}}_{\text{average of last gradients}}$$

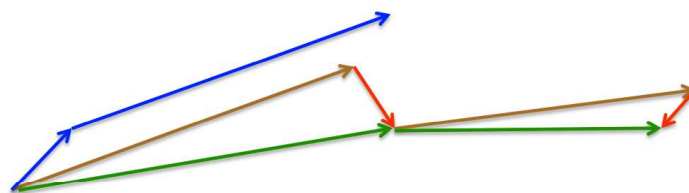
Typical value for $\gamma = 0.9$.

(2) Nesterov accelerated gradient

With momentum, we first compute the gradient and then make a step following our momentum and add the gradient. Nesterov proposed to first make the step following the momentum and then adjusting by computing the gradient locally:

$$v_{t+1} = \gamma v_t + \eta \nabla J(\theta_t - \gamma v_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$



Source for the image: [G. Hinton's lecture 6c](#)

(2) Adagrad

We would like to adapt our updates to each individual parameter, i.e. have a different decreasing learning rate for each parameter.

$$\begin{aligned}s_{t+1,i} &= s_{t,i} + \nabla J(\theta_t)_i^2 \\ \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{s_{t+1,i} + \epsilon}} \nabla J(\theta_t)_i\end{aligned}$$

(2) Adagrad

We would like to adapt our updates to each individual parameter, i.e. have a different decreasing learning rate for each parameter.

$$s_{t+1,i} = s_{t,i} + \nabla J(\theta_t)_i^2$$
$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{s_{t+1,i} + \epsilon}} \nabla J(\theta_t)_i$$

No manual tuning of the learning rate.

Typical default values: $\eta = 0.01$ and $\epsilon = 1e - 8$.

(2) RMSProp

Problem with Adagrad, learning rate goes to zero and never forgets about the past.

(2) RMSProp

Problem with Adagrad, learning rate goes to zero and never forgets about the past.

Idea proposed by G. Hinton in his coursera class: use exponential average.

$$s_{t+1,i} = \gamma s_{t,i} + (1 - \gamma) \nabla J(\theta_t)_i^2$$
$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{s_{t+1,i} + \epsilon}} \nabla J(\theta_t)_i$$

(2) RMSProp

Problem with Adagrad, learning rate goes to zero and never forgets about the past.

Idea proposed by G. Hinton in his coursera class: use exponential average.

$$s_{t+1,i} = \gamma s_{t,i} + (1 - \gamma) \nabla J(\theta_t)_i^2$$
$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{s_{t+1,i} + \epsilon}} \nabla J(\theta_t)_i$$

With a slight abuse of notation, we re-write the update as follows:

$$s_{t+1} = \gamma s_t + (1 - \gamma) \nabla J(\theta_t)^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_{t+1} + \epsilon}} \nabla J(\theta_t)$$

Typical values: $\gamma = 0.9$ and $\eta = 0.001$.

(2) Adam

Mixing ideas from RMSProp and momentum, we get Adam = Adaptive moment Estimation.

$$\begin{aligned}m_{t+1} &= \beta_1 m_t + (1 - \beta_1) \nabla J(\theta_t) \\v_{t+1} &= \beta_2 v_t + (1 - \beta_2) \nabla J(\theta_t)^2 \\\hat{m}_{t+1} &= \frac{m_{t+1}}{1 - \beta_1^{t+1}} \\\hat{v}_{t+1} &= \frac{v_{t+1}}{1 - \beta_2^{t+1}} \\\theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}} + \epsilon} \hat{m}_{t+1}\end{aligned}$$

$\hat{m}_t$ and $\hat{v}_t$ are estimates for the first and second moments of the gradients. Because $m_0 = v_0 = 0$, these estimates are biased towards 0, the factors $(1 - \beta^{t+1})^{-1}$ are here to counteract these biases.

(2) Adam

Mixing ideas from RMSProp and momentum, we get Adam = Adaptive moment Estimation.

$$\begin{aligned}m_{t+1} &= \beta_1 m_t + (1 - \beta_1) \nabla J(\theta_t) \\v_{t+1} &= \beta_2 v_t + (1 - \beta_2) \nabla J(\theta_t)^2 \\\hat{m}_{t+1} &= \frac{m_{t+1}}{1 - \beta_1^{t+1}} \\\hat{v}_{t+1} &= \frac{v_{t+1}}{1 - \beta_2^{t+1}} \\\theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}} + \epsilon} \hat{m}_{t+1}\end{aligned}$$

$\hat{m}_t$ and $\hat{v}_t$ are estimates for the first and second moments of the gradients. Because $m_0 = v_0 = 0$, these estimates are biased towards 0, the factors $(1 - \beta^{t+1})^{-1}$ are here to counteract these biases.

Typical values: $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e - 8$.

(2) AMSGrad

Sometimes, Adam forgets too fast, hence to fix it, we replace the moving average by a **max**:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \nabla J(\theta_t)$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) \nabla J(\theta_t)^2$$

$$\hat{v}_{t+1} = \max(\hat{v}_t, v_{t+1})$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}} + \epsilon} m_{t+1}$$

(3) PyTorch optimizers

All have similar constructor `torch.optim.*(params, lr=..., momentum=...)`. Default values are different for all optimizer, check the doc.

`params` should be an iterable (like a list) containing the parameters to optimize over. It can be obtained from any module with `module.parameters()`.

The `step` method updates the internal state of the optimizer according to the grad attributes of the params, and updates the latter according to the internal state.

```
criterion = nn.NLLLoss()
optimizer_vgg = torch.optim.SGD(model_vgg.classifier[6].parameters(), lr = 0.001)

def train_model(model, dataloader, size, epochs=1, optimizer=None):
    model.train()
    for epoch in range(epochs):
        for inputs, classes in dataloader:
            inputs = inputs.to(device)
            classes = classes.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, classes)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
```

The end.